```
        _;
    }
```

❶ Modifier for phases of the auction

❷ Modifier checking beneficiary

5.4.3 Privacy and security aspects

In a blind auction, during the `Bidding` phase, bidders place blind bids; thus, the contents of the bids (values) are private as well as secure. How do you guarantee privacy and security? You can achieve privacy by hashing the parameters. But although a hash may appear to human eyes to be indecipherable, it can be broken by brute force. The Keccak hash for the integer value (uint) 20, for example, is the same whether you're on the Earth, Moon, or Mars, in Lagos (Nigeria), or New York (United States), as follows:

```
0xce6d7b5282bd9a3661ae061feed1dbda4e52ab073b1f9285be6e155d9c38d4ec
```

This 32-byte hash value might look like gibberish to you, but a brute-force attack with knowledge of the approximate context and value of the auction item could easily decipher the bid value if so desired. So how do you secure it from such an attack? You can use a nonce or a secret password as the second parameter. This secret is like a personal identification number for your debit card. In this case, computing the Keccak hash of the value, packed up with the second secret password, makes it secure. These various forms of value 20 are shown in table 5.1. The first column is the plain data value, 20 (open); the second column is the Keccak256 hash of 20 (private but not secure); and the third column is the Keccak256 hash of 20 and the password (in this case, 0x426526) of choice from the decentralized participant (private and secure). In the table, the `abi.encodePacked()` function of Solidity creates a byte form of parameters before Keccak hashing. That's a simple hashing technique for privacy and security that you can apply in similar situations and, hence, another design principle that you can use in your applications to achieve these goals.

Table 5.1 Keccak hashing for privacy and security